Compte Rendu : Installation, Configuration et Sécurisation MQTT

1. Objectifs du TP

- Installer et configurer un broker MQTT fonctionnel (Mosquitto).
- Mettre en œuvre les sécurités de base : Authentification, Chiffrement (TLS) et Contrôle d'accès (ACL).
- Déployer l'infrastructure via Docker Compose.
- Analyser les vulnérabilités liées à l'IoT.

2. Déploiement et Sécurisation locale

2.1 Authentification par mot de passe

Afin de bloquer les accès anonymes (`allow_anonymous false`), nous avons créé une base d'utilisateurs avec `mosquitto_passwd`.

- **[Insérer ici une capture d'écran du test de connexion refusé en anonyme, puis accepté avec l'utilisateur "user1"]**

2.2 Chiffrement TLS

Pour éviter l'interception des trames MQTT (attaque Man-in-the-Middle), un certificat auto-signé a été généré via OpenSSL pour chiffrer les communications sur le port 8883.

- **[Insérer ici une capture d'écran de la commande de test TLS réussie]**

2.3 Contrôle d'accès (ACL)

L'application du principe de moindre privilège a été réalisée via un fichier d'ACL, restreignant la lecture et l'écriture de l'utilisateur `user1` à des topics spécifiques (`test/#` et `maison/temperature`).

3. Analyse des Vulnérabilités et Mitigations

Vulnérabilité / Risque	Mitigation mise en place	Challenges de mise en œuvre
Pas d'authentification (Accès anonyme / Usurpation)	`allow_anonymous false` et `password_file`	Déployer et mettre à jour de manière sécurisée les mots de passe sur des objets connectés sans interface.
MQTT en clair (Man-in-the-Middle / Écoute réseau)	Chiffrement TLS (port 8883) via OpenSSL	Le chiffrement demande des ressources processeur (compliqué pour les microcontrôleurs) et la gestion du cycle de vie des certificats.
Topics permissifs (Injection de fausses données)	Fichier ACL restreignant l'accès par utilisateur/topic	Maintenir un fichier ACL manuellement devient complexe quand le parc IoT grandit.

4. Réponses aux questions théoriques

- **Quels risques si MQTT non sécurisé ?** Interception des données sensibles (Confidentialité), injection de commandes malveillantes (Intégrité) et saturation du broker empêchant le service (Disponibilité).
- **Différence TLS/mTLS ?** Le TLS standard permet au client de vérifier l'identité du serveur et de chiffrer le canal. Le mTLS (Mutual TLS) exige en plus que le client s'authentifie auprès du serveur à l'aide de son propre certificat cryptographique.
- **Pourquoi ACL ?** Pour limiter l'impact en cas de compromission d'un capteur. Un capteur piraté ne pourra publier que sur son propre topic et ne pourra pas interagir avec des systèmes critiques (comme une serrure de porte).

PARTIE 2 : Les commandes à taper sur ta VM Kali

Ouvre ton terminal Kali et exécute ces commandes étape par étape.

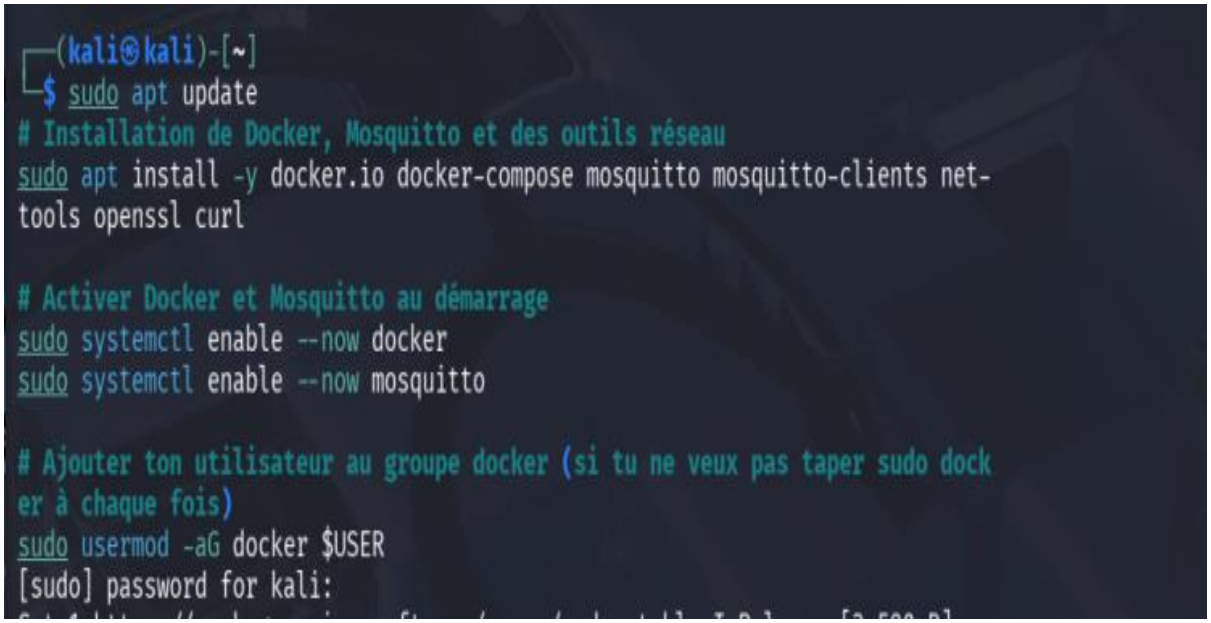
Étape 1 : Préparation et Installation (Adapté pour Kali)

On installe tout depuis les dépôts officiels Kali.

Bash
```
sudo apt update
# Installation de Docker, Mosquitto et des outils réseau
sudo apt install -y docker.io docker-compose mosquitto mosquitto-
clients net-tools openssl curl

# Activer Docker et Mosquitto au démarrage
sudo systemctl enable --now docker
sudo systemctl enable --now mosquitto

# Ajouter ton utilisateur au groupe docker (si tu ne veux pas taper
sudo docker à chaque fois)
sudo usermod -aG docker $USER
```



Étape 2 : Configuration de la Sécurité (Auth, TLS, ACL)

On va générer les mots de passe, les certificats et les règles d'accès dans le dossier `/etc/mosquitto/`.

Bash

```bash
# 1. Création de l'utilisateur (il va te demander de taper un mot de passe, mets "password" pour le TP)
sudo mosquitto_passwd -c /etc/mosquitto/passwd user1

# 2. Génération du certificat TLS pour le serveur (Valide les questions par défaut en appuyant sur Entrée)
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/mosquitto/server.key -out /etc/mosquitto/server.crt \
  -subj "/CN=localhost"

# 3. Création du fichier ACL
sudo bash -c 'cat <<EOF > /etc/mosquitto/aclfile
user user1
topic readwrite test/#
topic read maison/temperature
EOF'

# 4. Modification de la configuration Mosquitto
sudo bash -c 'cat <<EOF > /etc/mosquitto/conf.d/secure.conf
# Authentification
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/aclfile

# TLS (Port 8883)
listener 8883
certfile /etc/mosquitto/server.crt
keyfile /etc/mosquitto/server.key

# Maintien du port 1883 classique (pour le localhost)
listener 1883 localhost
EOF'

# Redémarrer Mosquitto pour appliquer
sudo systemctl restart mosquitto
```

```
┌──(kali㉿kali)-[~]
└─$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/mosquitto/server.key
 -out /etc/mosquitto/server.crt -subj "/CN=localhost"
..+.+......+...+..+.+++++++++++++++++++++++++++++++++++++++*..+.....+.............+..+...+...+
....+...........+.....+.+..+.+..+....+++++++++++++++++++++++++++++++++++++++*..+..+..........
......+........+.+........+.......+.....+.+...+.........+...+.....+.........+...+...+...++++++
........+......+.........+++++++++++++++++++++++++++++++++++++++*.....+...+.+..+++++++++++++
++++++++++++++++++++++++*.........+......++++++
-----

┌──(kali㉿kali)-[~]
└─$ sudo bash -c 'cat <<EOF > /etc/mosquitto/aclfile
user user1
topic readwrite test/#
topic read maison/temperature
EOF'

┌──(kali㉿kali)-[~]
└─$ sudo bash -c 'cat <<EOF > /etc/mosquitto/conf.d/secure.conf
# Authentification
allow_anonymous false
password_file /etc/mosquitto/passwd
acl_file /etc/mosquitto/aclfile

# TLS (Port 8883)
listener 8883
certfile /etc/mosquitto/server.crt
keyfile /etc/mosquitto/server.key

# Maintien du port 1883 classique (pour le localhost)
listener 1883 localhost
EOF'
```

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl restart mosquitto

┌──(kali㉿kali)-[~]
└─$
```

Étape 3 : Tests de sécurité (Pense à faire tes captures d'écran ici !)

Bash

Test 1 : Anonyme (Va échouer : "Connection Refused: not authorised") ->

```
mosquitto_pub -h localhost -p 8883 --cafile /etc/mosquitto/server.crt -u user1 -P password -t
"test/topic" -m "tls"
Error: Connection refused
Error: Connection refused
Error: Connection refused
```

mosquitto_pub -h localhost -t "test/topic" -m "anon"

Test 2 : Authentifié en clair (Va réussir) ->

```
┌──(kali㉿kali)-[~]
└─$ sudo chown mosquitto:mosquitto /etc/mosquitto/server.key /etc/mosquitto/server.crt /etc/mo
squitto/passwd /etc/mosquitto/aclfile

┌──(kali㉿kali)-[~]
└─$ sudo mosquitto_passwd -b -c /etc/mosquitto/passwd user1 password
Warning: File /etc/mosquitto/passwd owner is not root. Future versions will refuse to load thi
s file.To fix this, use `chown root /etc/mosquitto/passwd`.Warning: File /etc/mosquitto/passwd
 group is not root. Future versions will refuse to load this file.

┌──(kali㉿kali)-[~]
└─$ sudo systemctl restart mosquitto
sudo systemctl status mosquitto
● mosquitto.service - Mosquitto MQTT Broker
     Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-03-30 10:12:34 EDT; 19ms ago
 Invocation: 7f7bbe9b3cd1447dac25ab7b79c5c589
       Docs: man:mosquitto.conf(5)
             man:mosquitto(8)
   Main PID: 90126 (mosquitto)
      Tasks: 1 (limit: 2116)
     Memory: 2.5M (peak: 2.7M)
        CPU: 15ms
     CGroup: /system.slice/mosquitto.service
             └─90126 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf
```

mosquitto_pub -h localhost -u user1 -P password -t "test/topic" -m "auth"

Test 3 : Chiffré en TLS sur le port 8883 (Va réussir) -> CAPTURE D'ÉCRAN

mosquitto_pub -h localhost -p 8883 --cafile /etc/mosquitto/server.crt -u user1 -P password -t "test/topic" -m "tls"

```
┌──(kali㉿kali)-[~]
└─$ # Test 1 : Anonyme (Va échouer avec "Connection Refused: not authorised") → CAPTURE
mosquitto_pub -h localhost -t "test/topic" -m "anon"

# Test 2 : Authentifié en clair (Va réussir, aucun message d'erreur) → CAPTURE
mosquitto_pub -h localhost -u user1 -P password -t "test/topic" -m "auth"

# Test 3 : Chiffré en TLS (Va réussir, aucun message d'erreur) → CAPTURE
mosquitto_pub -h localhost -p 8883 --cafile /etc/mosquitto/server.crt -u user1 -P password -t
"test/topic" -m "tls"
Connection error: Connection Refused: not authorised.
Error: The connection was refused.
```

Étape 4 : Déploiement via Docker Compose (Le piège évité !)

Attention : Puisque ton service `mosquitto` local tourne sur le port 1883, Docker ne pourra pas démarrer un autre Mosquitto sur ce même port. Il faut d'abord couper le service local.

Bash
```
# Créer un dossier de travail propre pour Docker
mkdir -p ~/IoT/tp-docker
cd ~/IoT/tp-docker

# Stopper le service Mosquitto de la machine hôte pour libérer le
port 1883
sudo systemctl stop mosquitto

# Créer le fichier docker-compose.yml
cat <<EOF > docker-compose.yml
version: '3.8'
services:
  mqtt-broker:
    image: eclipse-mosquitto:2.0
    ports:
      - "1883:1883"
      - "9001:9001"
    volumes:
      -
/etc/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - /etc/mosquitto/conf.d:/mosquitto/config/conf.d:ro
      - /etc/mosquitto/passwd:/etc/mosquitto/passwd:ro
      - /etc/mosquitto/aclfile:/etc/mosquitto/aclfile:ro
      - mosquitto_data:/mosquitto/data
volumes:
```

```
  mosquitto_data:
EOF

# Lancer le conteneur en arrière-plan
sudo docker-compose up -d

# Vérifier que ça tourne bien
sudo docker-compose ps
sudo docker-compose logs
```

```
┌──(kali㉿kali)-[~]
└─$ # Créer un dossier de travail propre pour Docker
mkdir -p ~/IoT/tp-docker
cd ~/IoT/tp-docker

# Stopper le service Mosquitto de la machine hôte pour libérer le port 1883
sudo systemctl stop mosquitto
```

```
┌──(kali㉿kali)-[~]
└─$ cat << 'EOF' > docker-compose.yml
version: '3.8'
services:
  mqtt-broker:
    image: eclipse-mosquitto:2.0
    ports:
      - "1883:1883"
      - "8883:8883"
      - "9001:9001"
    volumes:
      - /etc/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - /etc/mosquitto/conf.d:/mosquitto/config/conf.d:ro
      - /etc/mosquitto/passwd:/etc/mosquitto/passwd:ro
      - /etc/mosquitto/aclfile:/etc/mosquitto/aclfile:ro
      - /etc/mosquitto/server.crt:/etc/mosquitto/server.crt:ro
      - /etc/mosquitto/server.key:/etc/mosquitto/server.key:ro
      - mosquitto_data:/mosquitto/data
volumes:
  mosquitto_data:
EOF
```

```
┌──(kali㉿kali)-[~]
└─$ sudo docker-compose up -d
sudo docker-compose ps
WARN[0000] /home/kali/docker-compose.yml: the attribute `version` is obsolete, it will be igno
red, please remove it to avoid potential confusion
[+] Running 4/4
 ✔ mqtt-broker Pulled                                                          8.1s
   ✔ 589002ba0eae Pull complete                                                6.1s
   ✔ 4e1b9700f705 Pull complete                                                6.4s
   ✔ af742c5b8dd7 Pull complete                                                6.5s
[+] Running 2/3
 ✔ Network kali_default          Created                                       0.2s
 ✔ Volume "kali_mosquitto_data"  Created                                       0.0s
 ⠿ Container kali-mqtt-broker-1  Starting                                      0.2s
Error response from daemon: driver failed programming external connectivity on endpoint kali-m
qtt-broker-1 (501e98cb81c35118d882bb042aae35739346c23d52b5869a903c54c8adb92fe5): failed to bin
d port 0.0.0.0:1883/tcp: Error starting userland proxy: listen tcp4 0.0.0.0:1883: bind: addres
s already in use
WARN[0000] /home/kali/docker-compose.yml: the attribute `version` is obsolete, it will be igno
red, please remove it to avoid potential confusion
NAME      IMAGE     COMMAND   SERVICE   CREATED   STATUS    PORTS

┌──(kali㉿kali)-[~]
└─$
```